

Software Development - Session 2

Python programming exercises

1. A perfect number is an integer whose sum of integer divisors (excluding the number itself) add up to the number. Following are the first four perfect numbers:

$$6 = 1 + 2 + 3$$

$$28 = 1 + 2 + 4 + 7 + 14$$

$$496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248$$

$$8128 = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 127 + 254 + 508 + 1016 + 2032 + 4064$$

Numbers whose sum of divisors is larger than the number are called abundant numbers. Those whose sum is less than the number are called deficient numbers. Perfect numbers have other interesting properties, but for now we are interested only in writing a program that sorts integers into one of three categories: perfect, abundant, or deficient. Write a program so that we get the following results:

```
>>>
Please enter a number to check:100
100 is not perfect
>>> ===== RESTART =====
>>>
Please enter a number to check:128
128 is not perfect
>>> ===== RESTART =====
>>>
Please enter a number to check:496
496 is perfect
```

2. Continuing on task 1, we would like to check a whole range of number and decide what each number is: abundant, deficient, or perfect. In this way we can look for more perfect numbers. Write a program so that we get the following results:

```
>>>
What is the upper number for the range:29
2 is deficient
3 is deficient
4 is deficient
5 is deficient
6 is perfect
7 is deficient
8 is deficient
9 is deficient
10 is deficient
11 is deficient
12 is abundant
13 is deficient
14 is deficient
15 is deficient
16 is deficient
17 is deficient
18 is abundant
19 is deficient
20 is abundant
21 is deficient
22 is deficient
23 is deficient
24 is abundant
25 is deficient
26 is deficient
27 is deficient
28 is perfect
29 is deficient
```

3. Write a program that plays a “hi-low” number guessing game. The program starts by generating a random number hidden from the user that is between 0 and 100. The user then attempts to guess the number, getting hints as to which direction (bigger or smaller, higher or lower) to go on the next guess. The game can end in one of two ways:

- The user can correctly guess the number.
- The user can quit playing by entering a number out of the range of 0–100.

4. Write a program to generate the hailstone sequence which is formed by applying the formula below to the initial number and then repeatedly to each number generated by the formula. The result is a sequence of integers ending at 1. For example, if you start with 5, you get the following sequence: 5, 16, 8, 4, 2, 1. The hailstone formula is as follows:

- If the number is even, divide it by 2.
- If the number is odd, multiply by 3 and add 1.
- When the number reaches 1, quit.

5. Consider an experiment of tossing a coin three times. Write a program to print out all possible outcomes of the experiment with H: head and T: Tail.
6. Consider an experiment of drawing two cards at random from a bag containing four card marked with the integer 1 through 4. Write a program to:
 - a. Print all possible outcomes of the experiment if the first card is replaced before the second is drawn.
 - b. Print all possible outcomes of the experiment if the first card is not replaced
7. Consider an experiment of tossing two dice. Write a program to:
 - a. print out all possible outcomes of the experiment as (x, y)
 - b. print out all possible outcomes that the sum of the dots on the dice equals 7